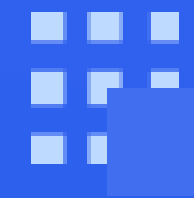




软件测试

Bug的反向定义



回顾：软件Bug的定义

Bug在程序的不同阶段，分别称为Fault, Error和Failure，定义如下：

- Fault-故障：是指静态存在于程序中的缺陷代码；
- Error-错误：是指程序运行缺陷代码后导致错误的程序状态；
- Failure-失效：是指程序错误状态传播到外部被感知的现象。

思考与讨论

- 实际应用中Fault Error, Failure的三者的关系？
- 实际应用中Fault Error, Failure观察难易程度？

深入分析Bug ■ Bug的反向定义



Fault是代码中的缺陷，通常需要经验丰富的程序员通过代码审查发现。

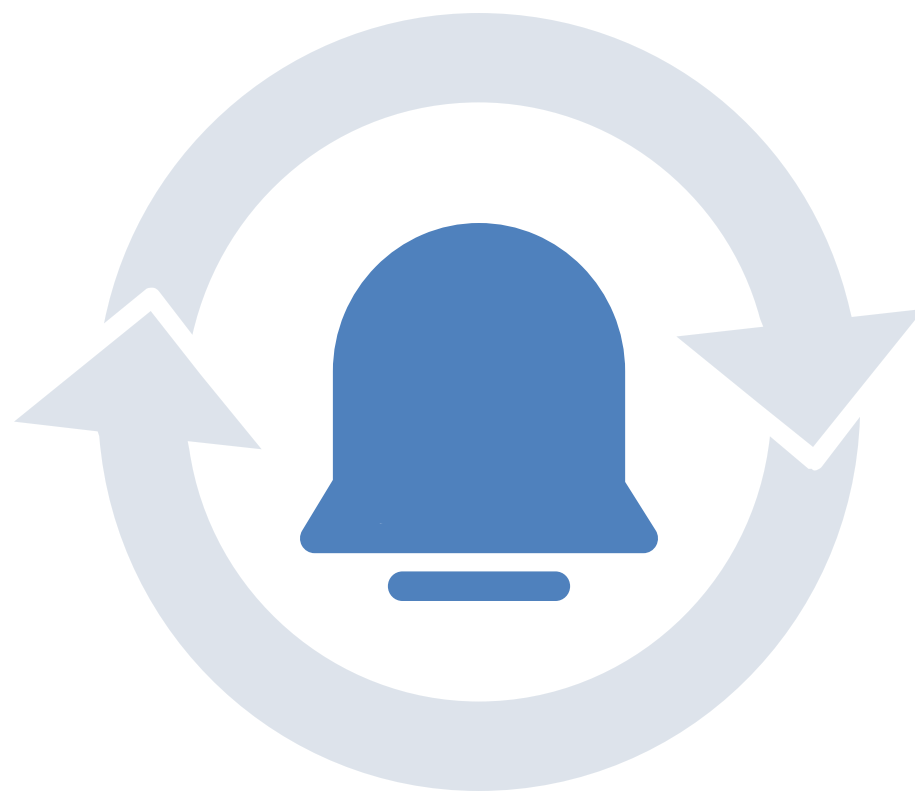


Error是程序运行时的异常状态，需要监控程序运行信息进行分析。



Failure 是程序的实际输出与预期输出不符，测试人员通过对比判定。

应用实践



一个循环的起始条件错误，可能导致程序逻辑混乱，难以直接察觉。

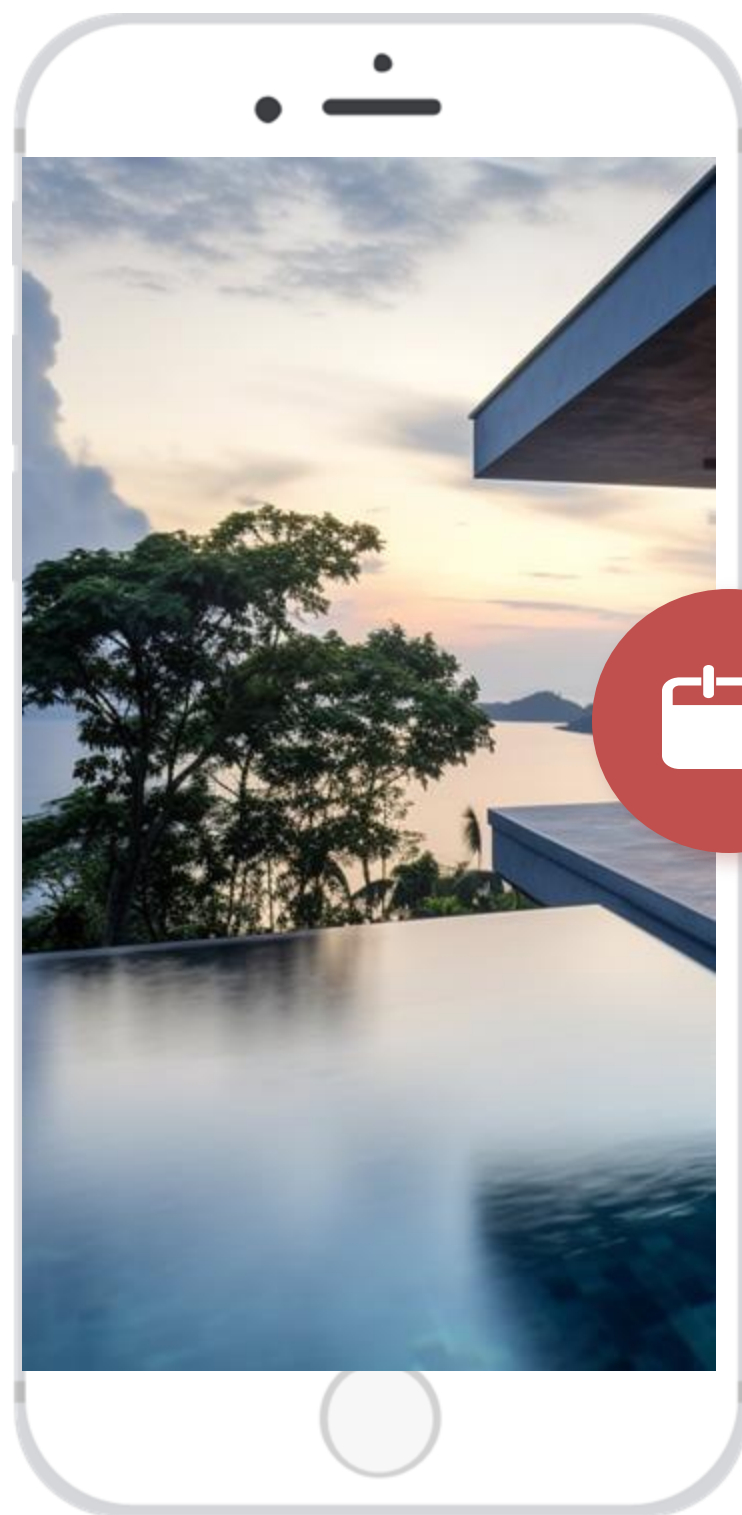


计算过程错误，程序运行时状态出错，需要查看日志定位问题。



计算平均值程序，输入 [4, 3, 5]，预期输出 4，实际输出 3.5。

反向定义概述



Bug的正向定义

给定一个完全正确的程序 P 和测试 t ，若对于 P 的修改版本 P' 若 $P'(t)$ 失效，则称程序修改部分 $P' \setminus P$ 为故障 (Fault)。

通过失效的消失确认故障位置

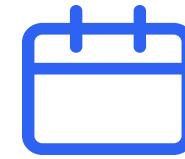
在工程实践中，通常无法预先得到一个正确的程序，而是通过逐步测试和修复来接近正确的版本。Bug的反向定义是通过程序失效 (Failure) 的消失和程序修复来确认故障 (Fault) 的位置。

Bug的反向定义

给定程序 P 和测试 t ，若 $P(t)$ 失效，修改后得到新程序 P' ，若 $P'(t)$ 通过，则确认程序修改部分 $P \setminus P'$ 为故障 (Fault)。

深入分析Bug ■ Bug的反向定义

```
void MeanVar(double X[], int L) {  
    double mean, sum;  
    sum = 0;  
    for (int i = 1; i < L; i++) { // 修复方案1:  
i=0  
        sum += X[i];  
    }  
    mean = sum / L; // 修复方案2:  
mean=sum/(L-1)  
    printf("Mean: %f\n", mean);  
}
```



代码中的Fault

程序代码第4行代码的起始下标错误导致失效。
初级程序员可能误以为第7行代码是问题所在。



测试数据与问题分析

测试数据：输入 [4, 3, 5]，预期输出为 4。
问题分析：第 4 行代码的起始下标错误导致失效，实际输出为 3.5。



修复方案与风险

修复方案 1：将第 4 行的 $i = 1$ 修改为 $i = 0$ 。
修复方案 2：将第 7 行的 L 修改为 $L - 1$ 。
风险：
修复依赖于测试集和修复方式，具有较大风险。
程序员可能错误地修复问题，导致新的 Bug。

01

反向定义的应用

实践中，Bug 检测常采用反向定义，通过失效的消失确认故障位置。这种方法依赖测试和修复过程，能够有效定位和修复问题。

02

修复过程的风险

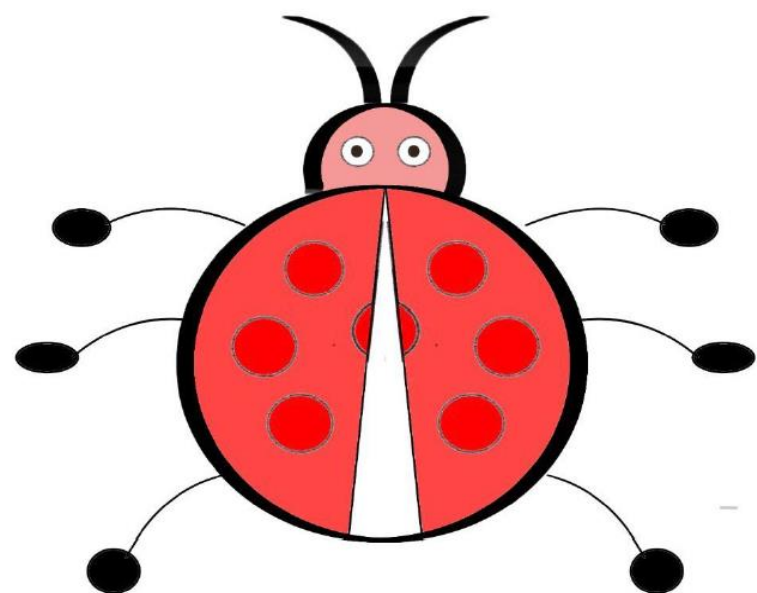
修复过程依赖测试集和修复方式，存在一定风险。初级程序员可能错误地修复问题，导致新的 Bug。

03

工程中的修复方法

工程中结合缺陷跟踪、修复块分析等方法，提高修复的准确性和效率。这些方法能够帮助更好地管理和优化修复过程。

回顾与讨论



反向定义流程

- 执行测试
- 失效分析
- 缺陷定位
- 缺陷修复
- 测试确认

潜在风险

- 不同修复方法带来不同的Bug
- 不同测试确认带来不同的Bug